TWIN-64
Architecture Document

Helmut Fieres
July 27, 2025

Contents

Introduction

Designers of the seventies and early eighties CPUs almost all used a micro-coded approach with hundreds of instructions. RISC was just starting to enter the stage. Registers were typically not really generic but often had a special function or meaning and many instructions were rather complicated and the designers felt they were useful. The compiler writers often used a small subset ignoring these complex instructions because they were so specialized. The later eighties and nineties shifted to RISC based designs with large sets of registers, fixed word instruction length, a large virtual memory address range and instructions that are in general much more pipeline friendly. What if some of these principles had found their way earlier into CPU designs?

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a register-memory model and segmented virtual memory. The design is heavily influenced by Hewlett Packard's PA_RISC architecture, which was initially a 32 bit RISC-style register-register load-store machine. Almost all of the key architecture features come from there. However, other processors, such as the Data General MV8000, the DEC Alpha, and the IBM PowerPc, were also influential or at least investigated for their architectural design choices. The original design goal that started this work was to truly understand the design process and implementation tradeoffs for a simple pipelined CPU.

While it is not a design goal to build a modern, competitive CPU, the CPU should nevertheless be useful and largely follow established common design practices. For example, instructions should not be invented because they seem to be useful, but rather designed with the assembler and compilers in mind. Instruction set design is also CPU design. Register memory architectures were typically micro-coded complex instruction machines. In contrast, VCPU-32 instructions will be hard coded and are in general "pipeline friendly", avoiding data and control hazards and stalls where possible.

Concepts

This section outlines the guiding principles and concepts chosen for Twin-64.

The instruction set design guidelines center around the following principles. First, in the interest of a simple and efficient instruction decoding step, instructions are of fixed length. A machine word is the instruction word length. As a consequence, some address offsets are rather short and addressing modes are required for reaching the entire address range. Instead of a multitude of addressing modes, typically found in the CISC style CPUs, Twin-64 offers very few addressing modes with a simple base address - offset calculation model. No indirection or any addressing mode that would require to read a memory item for address calculation is part of the architecture.

There will be few instructions in total, however, depending on the instruction several options for the instruction execution are encoded to make an instruction more versatile. For example, a boolean instruction will offer options to negate the result thus offering and "AND" and a "NAND" with one instruction. Wherever possible, useful instruction options such as the one outlined before are added to an instruction, such that it covers a range of instructions typically

found on the CPUs looked into. Great emphasis is placed in that such options do not overly complicated the pipeline and only increase the overall data path length slightly.

Modern RISC CPUs are typically load/store CPUs and feature a large number of general registers. Operations takes place between registers. Twin-64 follows a slightly different route. There is a smaller number of general registers and in addition to computation between registers, a register-memory model is also supported. There are however no instructions that read and write to memory in one instruction cycle as this would complicate the design considerably.

Twin-64 offers a large address range, organized in segments. Segment Id and offset from a virtual address. In addition, a short form of a virtual address, called a logical address, will select segment register based on the upper two bits of the logical address to form a virtual address. Segments are also associated with access rights and protection identifies. The CPU itself can run in user and privilege mode.

This document describes the overall architecture, instruction set and runtime model for Twin-64. It is organized into several parts. The first part will give an overview on the architecture. It presents the memory model, registers sets and basic operation modes. The major part of the document then presents the instruction set. Memory reference instructions, immediate instructions, branch instructions, computational instructions and system control instructions are described in detail. These chapters are the authoritative reference of the instruction set.

The runtime environment chapters present the runtime architecture. Although the CPU architecture and instructions allow for a great flexibility how to write programs, a runtime architecture is essential to define the common usage of the instruction set for assembler and compilers.

This book

This book defines the overall Twin-64 architecture. It is organized into several chapters.

Architecture Overview

64 bit architecture

only signed 64 bit math

multi processor

processors have a TLB and a cache

52 bit virtual memory

34 bit physical memory

I/O is memory mapped

I/O Modules - System Bus

instructions feature register memory and load / store concept

register offset and register indexed, scalable

System Organization

Data Types

Twin-64 is a big endian 64-bit machine. The fundamental data type is a 64-bit signed integer with the most significant bit being left. Bits are numbered from left to right, starting with bit 64 as the MSB bit down to bit 0 as LSB. Memory access is performed on a double, word, half-word or byte basis. All addresses are however always expressed as bytes addresses.

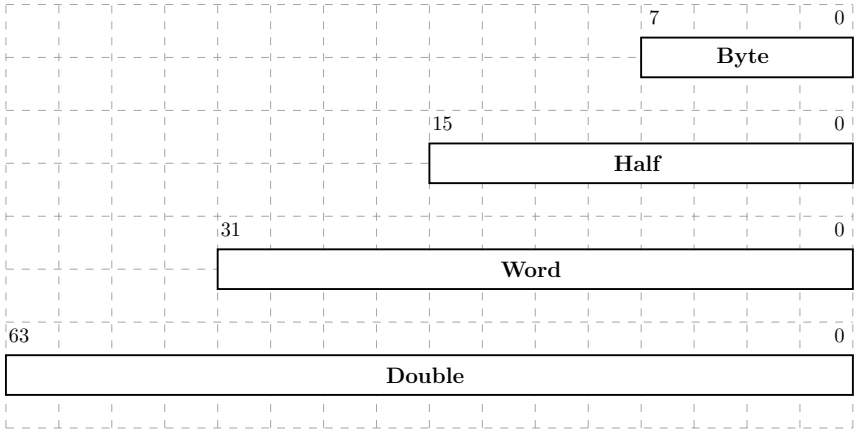


Figure 1: Data types

lower sizes are sign extended before use

address computation uses a 32-bit unsigned math, i.e. numbers wrap around in 32 bit space.

Byte Ordering

Twin-64 supports little endian and big endian byte ordering models.

General Registers

Twin-64 features a set of 16 general purposes registers.

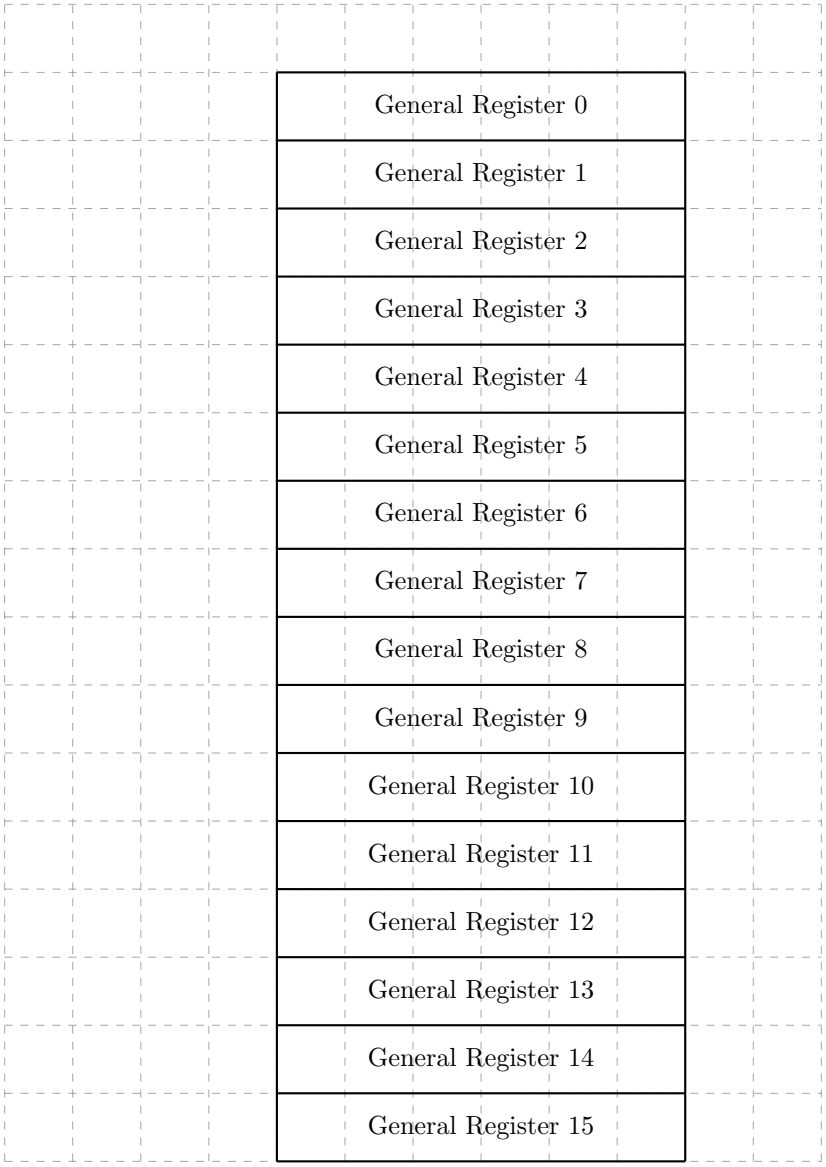


Figure 2: General registers

Control Registers

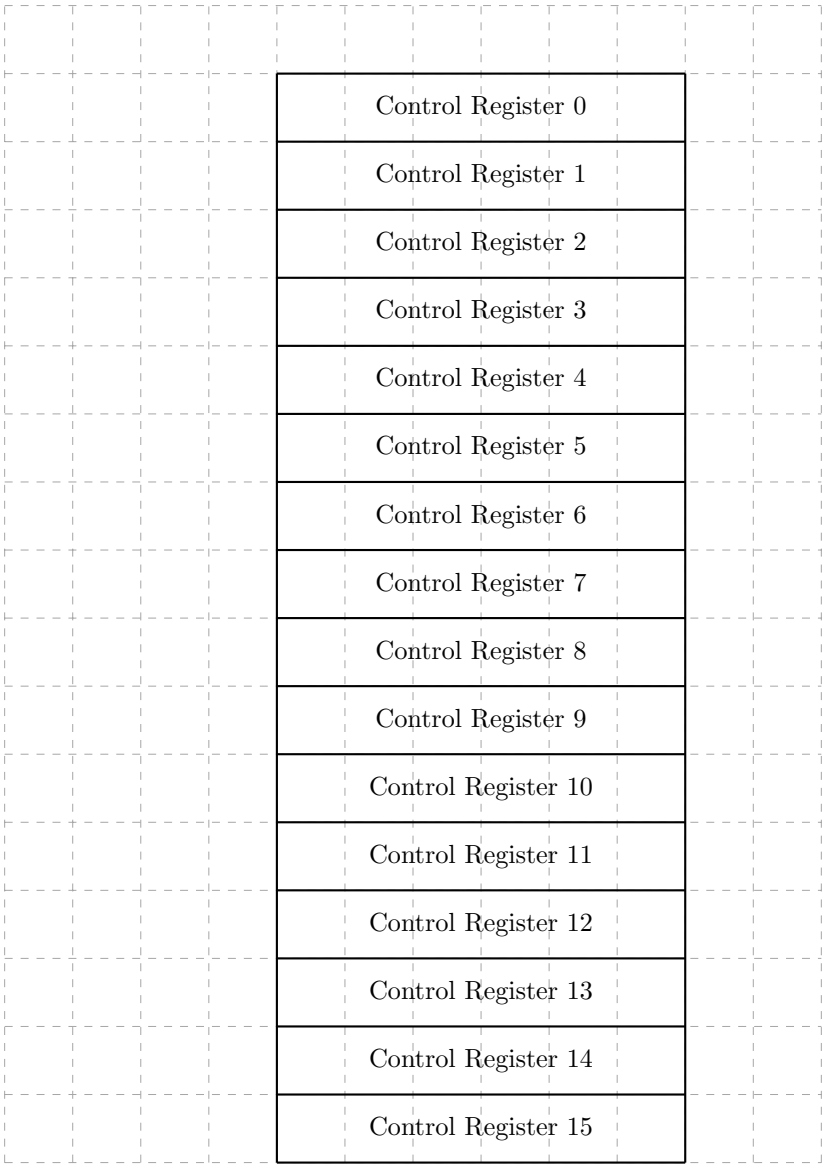


Figure 3: General registers

Some general registers have a dedicated use. Register zero will always return a zero value when read and a write operation will have no effect. Although there are only 16 general registers in the CPU, implementing such a register greatly simplifies the instruction design, in that it for example provides a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for some instructions. Other general registers may have dedicated purpose defined in the runtime architecture.

Program State Register

Figure 4: Program state register

TLB

fully associative TLB

support for large pages (16Kb, 256 Kb, 4Mb, 64 Mb)

implementations can choose a unified or a split TLB

bits:

V - valid

L - entry locked

M - modified trap

B - branch taken trap

Caches

cache visible architecture

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range. The address range is organized into a 20-bit segment identifier and a 32-bit offset into that segment. The following figure shows the virtual address and how the segment is selected.

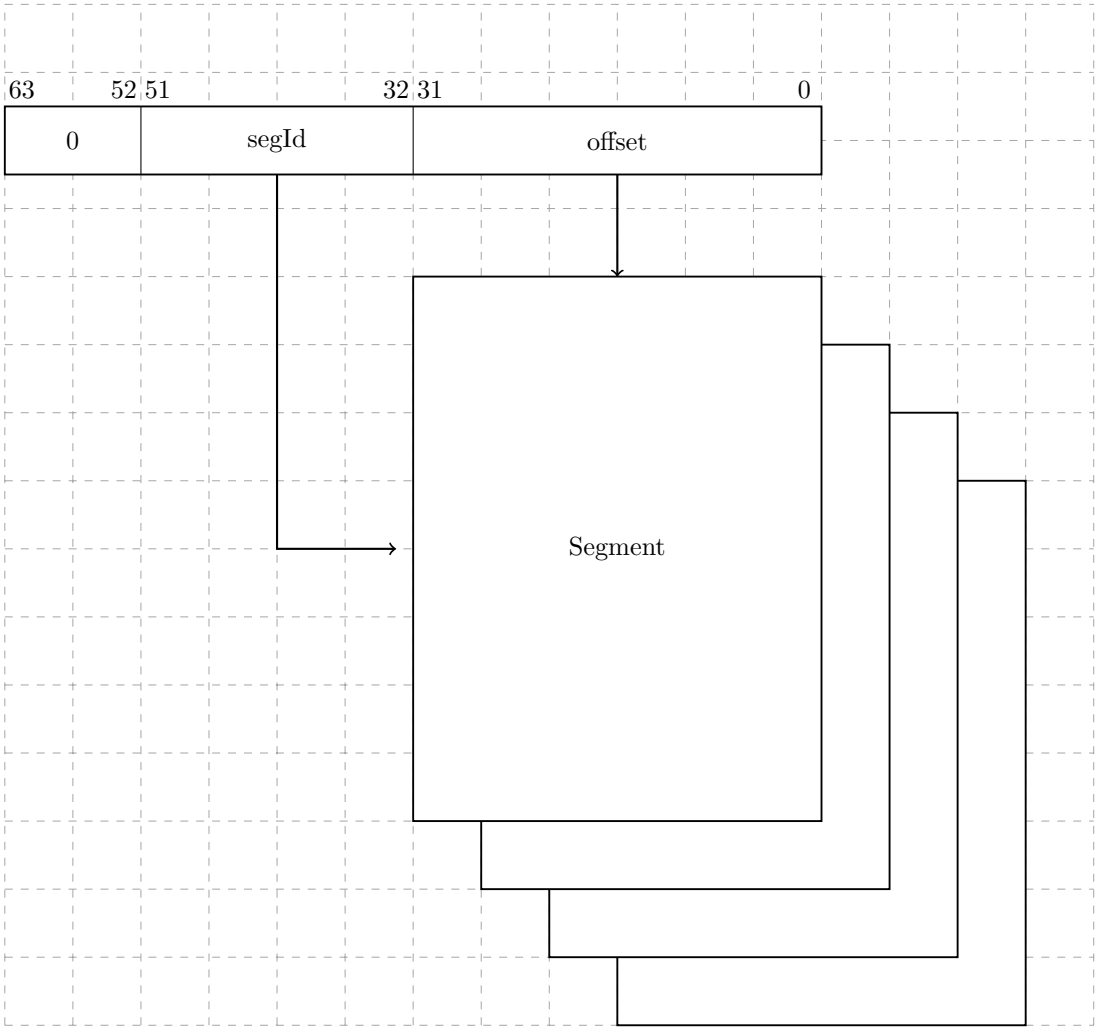


Figure 5: Virtual Memory Address Range

A virtual address to be translated to a physical address. For address translation purposes, the virtual address range can also be seen as a set of virtual pages. The architected page size is 16Kb. A virtual page number is a 38-bit page number with a 14-bit offset into that page.

Physical Address Space

Twin-64 architecture features a 34-bit physical memory address range which allows a maximum physical memory of up to 16 GBytes.

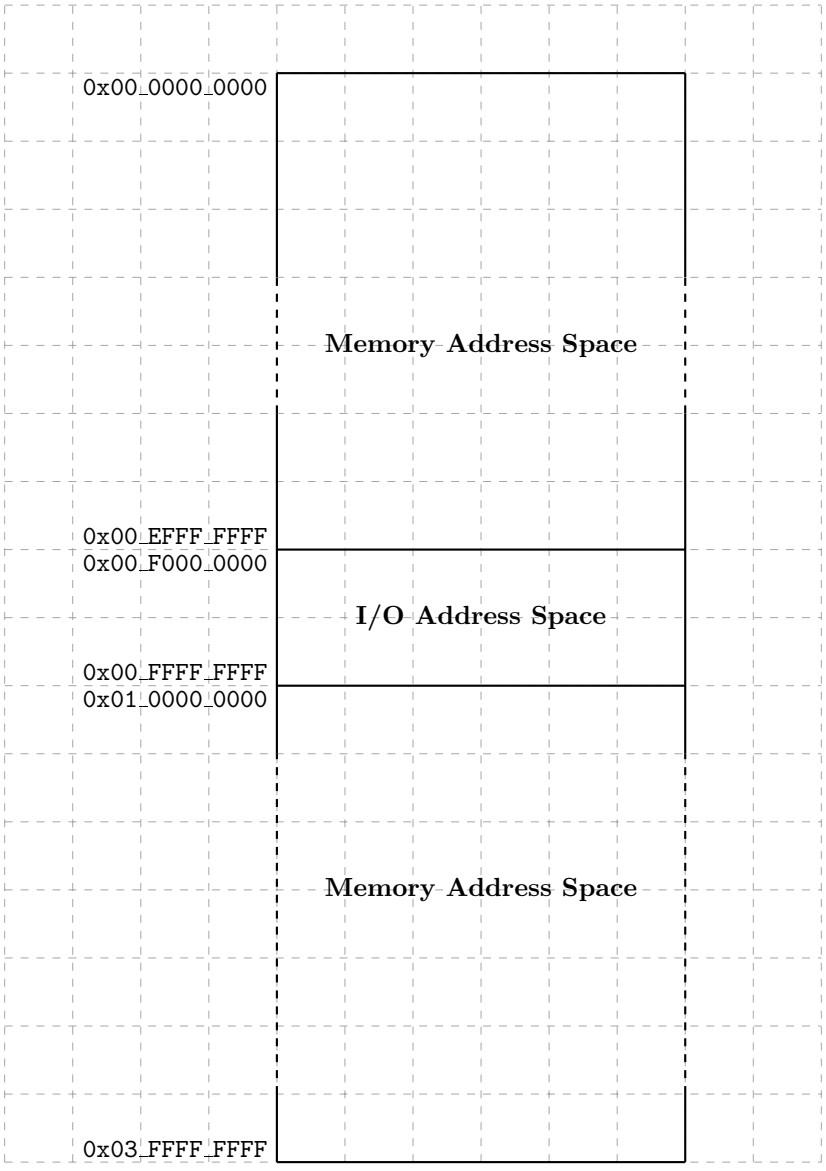


Figure 6: Physical Memory Address Range

segment Id 0 to 3 directly map to the physical address, without TLB translation and access rights checking. Only privileged mode can access these segments.

I/O Address Space

Twin-64 features memory mapped I/O architecture. As shown in the overall physical memory layout, the physical memory address range is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges in more detail.s

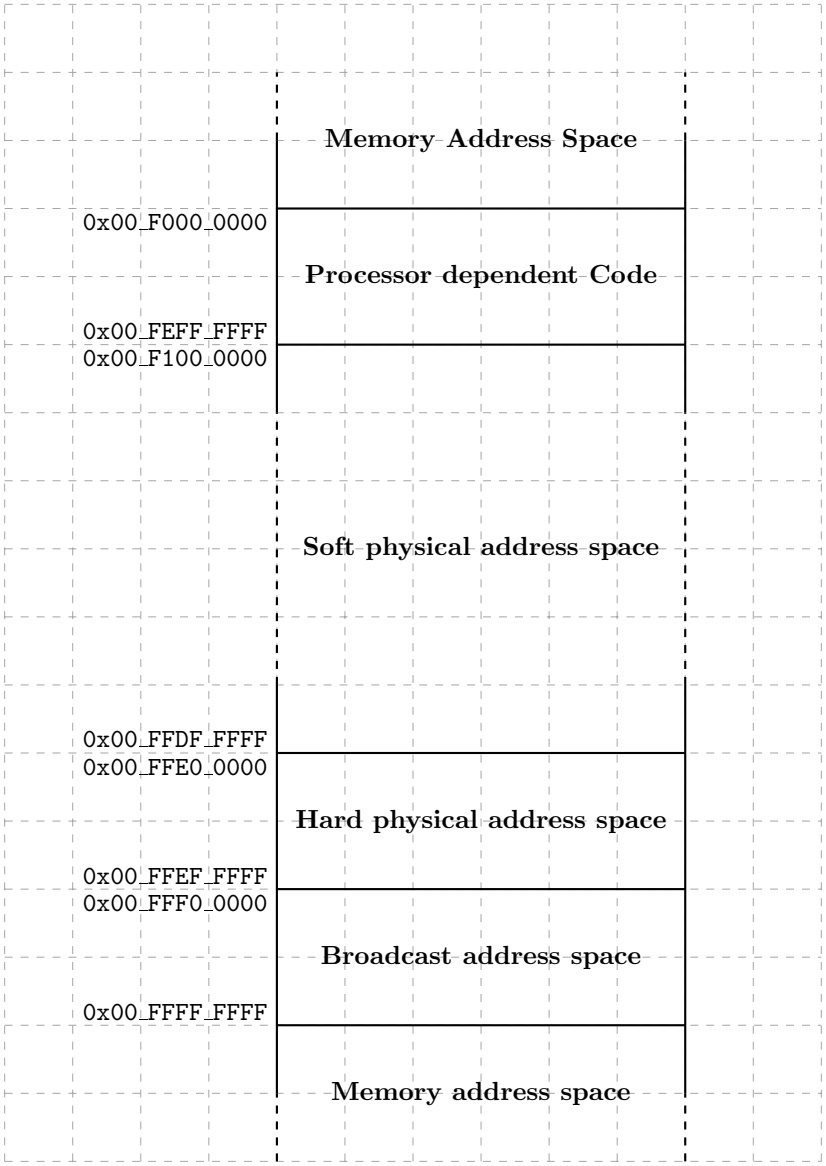


Figure 7: I/O Memory Address Range

always uncached

The I/O address space dedicates 256 Mbytes to an area that contains the processor dependent code. Typically, this is a set of library functions to manage the processor itself but also provide low-level primitives for the boot process and the operating system. The remainder of the I/O address space is divided into I/O modules, which map the I/O hardware components to a memory addressable location. Both PDC and I/O firmware design is explained in a later part of the document.

Addressing and Access Control

Address Translation

The Twin-64 CPU emits always a virtual address. Each memory access needs to be translated into a physical address.

virtual address for the first four segments directly map to the physical address space.

Access Control

on page level

Access Protection

on page level

Interruptions

Traps

External Interrupts

Input/Output

Concepts

memory mapped

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

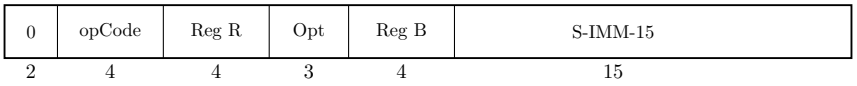
memory module has entire memory as SPA space

Instruction Set

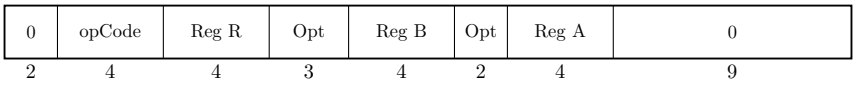
Instruction Overview

Arithmetic Instructions

The arithmetic instruction operate on two signed 64-bit operands and stores the result to the target register. There are two instruction layouts, immediate and register based. The **immediate operand instruction format** will add the sign extended 15-bit value to `RegB`. The register instruction format instruction will add `RegA` and `RegB`. The following figure shows the instruction layout.



The **register instruction format**type instruction uses two registers as input operands, performs the operation and stores the result in a third register. The following figure shows the instruction layout.



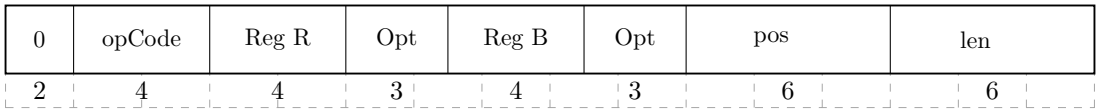
The **ADD** and **SUB** perform signed integer 64-bit arithmetic with an overflow resulting in an overflow trap. There are no operations on unsigned quantities. The **SHLxA** and **SHRxA** perform an arithmetic shift operations and add an operand to the shifted input, storing the result in the target register.

The **CMP** compares two registers for being equal, not equal, less than and les or equal than. The result of this comparison is store din the target register.

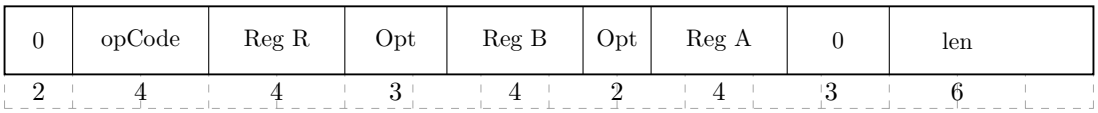
Bit Operations Instructions

The bit operation instruction fall into two groups. The **AND**, **OR** and **XOR** instructions implement the logical operations as their name suggest. Like the arithmetic instructions, there is an immediate instruction and a register instruction layout.

The second group implements the bit manipulation operations. The **EXTR** instruction extracts a bit field from a register and places the field in another register. Optionally, the extracted field is signed extended. The **DEP** instruction deposits a bit field in a target register.



The **DSR** instruction concatenates two general registers, shifts the 128-bit quantity to the right and stores the lower 32 bits in a target register.



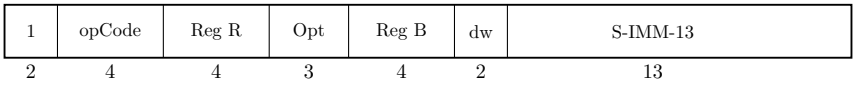
Immediate Instructions

LDI, ADDIL, LDO

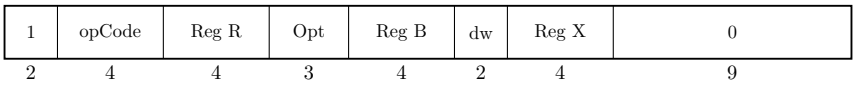
Memory Reference Instructions

T64 is a register memory architecture. It has the common load/store instructions as well as instructions that combined an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register and offset instruction. the second the base register and register index instruction.

The **indexed instruction format** will load the content of memory address formed by adding the scaled immediate 13-bit field to the base register `RegB` and add the data value to `RegA`. The **dw** field indicates the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. In addition, the **dw** field will scale the offset value by left shifting the immediate field according to the data width. The following figure shows the general instruction layout for base register and offset instruction.



The **register indexed instruction format** will load the content of memory address formed by adding `RegX` to the base register `RegB` and add the data value to `RegA`. Both load formats will use the **dw** field to specify the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. The offset in `RegX` is interpreted as a byte offset, independent of the actual **dw** field value. The following figure shows the general instruction layout for base register and register index type instructions.



The **LD** and **ST** instruction are the load and store instructions. In addition to the two addressing modes presented above, they also feature a base register address modification. Depending on the offset sign, the base register is updated before or after the address computation with the effective address computed. See the instruction description for more details.

The **LDR** and **STC** instruction implement the base support for a pipeline friendly method for semaphore operations. They only support the base register and offset mode. Also, there is no base register modification option.

The **ADD**, **SUB**, **AND**, **OR**, **XOR**, **CMP** instructions perform the respective operation as described above with one operand being fetched from memory.

Branch Instructions

B, BR, BV, BB, CBR, MBR

System Instructions

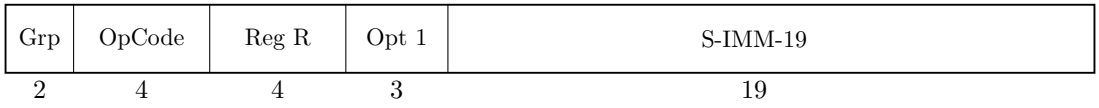
MFCR, MTCR, RSM, SSM, RFI, ITLB, PTLB, PCA, FCA, LDPA, PRBR, PRBW

Instruction Formats

T64 uses few instruction formats. They are grouped by the number of available register slots and the length of the immediate value field.

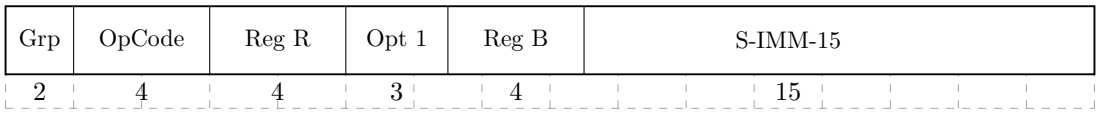
Immediate S19

The immediate S19 instruction format provides one register and a 19 bit signed immediate field. This format is typically used by branch instructions.



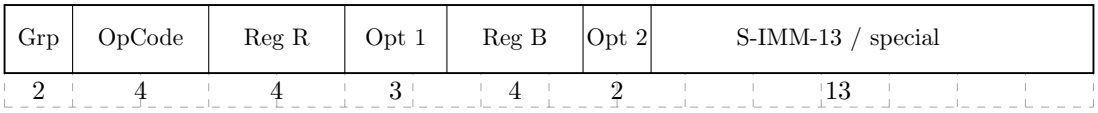
Immediate S15

The immediate S15 instruction format provides two registers and a 15 bit signed immediate field. This format is used by branch instructions as well as instruction which operate on a register and an immediate value.



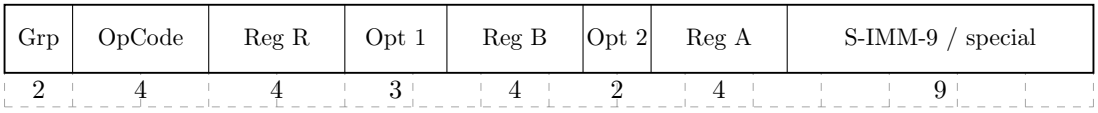
Immediate S13

The immediate S13 instruction format provides two registers, an additional option field and a 13 bit signed immediate field. Some instruction use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions where the address is formed by a base register and a 13-bit signed offset.



Immediate S9

The immediate S9 instruction format provides three registers, an additional option field and a 9 bit signed immediate field. Some instruction use the S-IMM-9 field for special encodings instead of a signed value.instead of a signed value. This instruction format is used for all computational instructions where three registers are needed.



Immediate U20

The immediate U20 format is used by the immediate instruction group to provide a 20bit unsigned value. This value is then placed at certain positions in the register target Reg R.

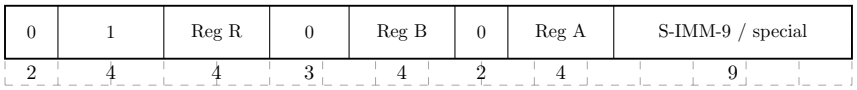
ADD Integer Addition

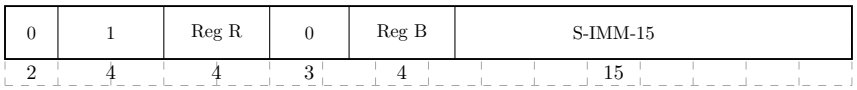
Syntax

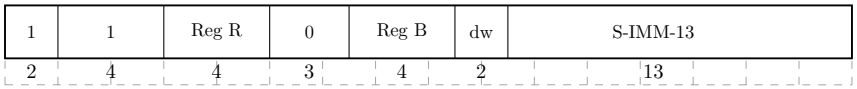
```
ADD RegR, RegB, RegA
ADD RegR, RegB, Immed15
ADD RegR, Immed13( RegB )
ADD RegR, RegX ( RegB )
```

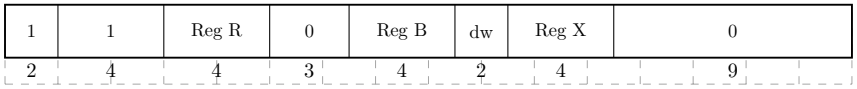
Format

The ADD instruction uses the register, the immediate, the indexed and the register indexed instruction formats.









Description

The `ADD` instruction adds two signed 64-bit operands and stores them to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats.

Operation

`RegR <- RegB + RegA` (register format)
`RegR <- RegB + immOperand( )` (immediate format)
`RegR <- RegR + memOperand( )` (indexed formats)

Exceptions

Integer Overflow
Data Alignment
Memory Access Violation
Memory Protection Violation
Data TLB miss

Notes

None.